# *Morse Code Encoder and Decoder Using Microcontrollers and C Code*

Team Name: The Big Sharks!
Noble Carpenter, Teagan Tobias
ECE 3130, Microcomputer Systems, Tarek Elfouly

Submission Date: April 24th, 2026

# Table of Contents

Table of Contents…… 2
I. INTRODUCTION…… **3**
II. PROJECT SPECIFICATIONS AND DESCRIPTION…… **4**
A. Project Description…… 4
B. Block Diagram…… 4
III. DETAILED IMPLEMENTATION…… **5**
A. Program Structure…… 5
B. Interface Design…… 9
C. Microcontroller Resource Utilization…… 10
D. Software…… 11
E. Hardware Photos…… 12
IV. ANALYSIS…… **13**
A. Functionality and Testing…… 13
B. Required Discussion Questions…… 14
V. DESCRIPTION OF TEAMWORK EXPERIENCE…… **15**
A. Collaborative and Inclusive Environment…… 15
B. Leadership and Goal Setting…… 15
C. Task Planning and Distribution…… 15

# I. INTRODUCTION

This project implements a two-mode Morse code terminal on the STM32 NUCLEO-L476RG microcontroller paired with the EDUBase V2 development board. In Encode mode, the user types alphabetic characters on the 4x4 matrix keypad using a multi-tap (T9-style) input scheme, and the system outputs the corresponding International Morse code patterns through a buzzer and LED indicator. In Decode mode, the user taps the blue NUCLEO push button (short press for dit, long press for dah) and the system measures the press duration using a hardware timer, accumulates the pattern, and displays the decoded letter on the LCD and 7-segment display. Mode switching between encode and decode is accomplished by pressing the asterisk key on the keypad.

The real-world relevance of this project extends to emergency communication, amateur radio operation, and accessibility technology. Morse code remains an internationally recognized distress signal (SOS) and is still used in maritime and aviation contexts. A hardware Morse terminal demonstrates the fundamental embedded system skills, such as: GPIO interfacing, timer-driven output, interrupt-driven input, and state machine architecture. These skills underpin virtually all modern embedded control systems.

The team successfully completed both encode and decode modes with full interrupt-driven architecture, hardware timer integration, multi-tap letter input covering all 26 letters and 10 digits, and error handling. Six different peripheral types (keypad, LCD, 7-segment display, buzzer, LEDs, and push buttons) were integrated and functional. The primary challenge encountered was coordinating three concurrent hardware timers (TIM2, TIM3, TIM6) without any blocking delays in the main loop or ISRs. A second significant challenge was hardware conflicts between the EDUBase push buttons and the keypad row pins (which share the same GPIO port), resolved by implementing mode toggle entirely in software via the '*' keypad key.

This report is organized as follows: Section II provides the project specifications and a block diagram. Section III covers the detailed implementation including pseudocode, FSM flowcharts, interface design, and microcontroller resource utilization. Section IV presents the testing methodology and results alongside discussion questions. Section V discusses teamwork experience.

# II. PROJECT SPECIFICATIONS AND DESCRIPTION

## A. Project Description

The system operates in one of the two modes at any given time, selectable by pressing the '*' key on the keypad at any moment:

- Encode Mode:
  - The user types letters and digits using a multi-tap (T9-style) scheme. Digit keys 1-7 cycle through letter groups plus the digit itself (e.g. key 1 cycles E/F/G/1). Keys A-D produce those letters directly. Keys 0, 8, and 9 produce those digits directly. Selections confirm automatically after approximately one second of inactivity, or immediately when a different key is pressed. The '#' key inserts a 7-tick word gap. The system looks up the Morse pattern and plays it back through the buzzer and LED using hardware-timed durations. A circular queue allows characters to be typed ahead while a previous character is still playing.
- Decode Mode:
  - The user taps the blue NUCLEO button (PC13). EXTI interrupts on both edges capture press start (falling edge, pin goes LOW) and release (rising edge, pin goes HIGH). TIM3 measures the press duration in milliseconds. Presses shorter than 300 ms are classified as dots; longer presses as dashes. After 600 ms of silence enforced by TIM6, the accumulated dot/dash pattern is looked up in the Morse table and the matching letter is displayed.

The Morse timing standard used is the International Morse Code convention where one dot unit equals 200ms (one TIM2 tick), dashes last 600ms (three ticks), inter-symbol gaps are 200ms, inter-letter gaps are 600ms, and inter-word gaps are 1400ms (seven ticks).

## B. Block Diagram

The system architecture is shown conceptually below. The STM32 Cortex-M4 processor is at the center, connected to all peripherals through GPIO ports A, B, and C.

| INPUT DEVICES | STM32 L476RG<br>ARM Cortex-M4 | OUTPUT DEVICES |
|---|---|---|
| 4x4 Keypad (PB1-4, PB8-11)<br>PC13 Blue Button (dot/dash)<br>'*' key = mode toggle<br><br>SW2 also mode toggles. | TIM2: 200ms encode tick<br>TIM3: press measurement<br>TIM6: 600ms decode timeout<br>EXTI13: PC13 both edges<br>NVIC: interrupt controller | LCD 2x16 (PB5, PA5, PA10)<br>7-Segment x4 (PB5, PA5, PC10)<br>Buzzer (PC9)<br>LEDs x4 (PA1, PA0, PC7, PC8) |

# III. DETAILED IMPLEMENTATION

## A. Program Structure

**Pseudocode – Encode Mode**

```
INIT: enable clocks GPIOA/B/C, TIM2/3/6, EXTI13
INIT: configure all GPIO pins (MODER, OTYPER, PUPDR)
INIT: LCD reset sequence, keypad column/row setup
TIM2_Start()  // 200 ms periodic ISR begins

MAIN LOOP (encode mode):
        confirmed = Keypad_Read()       // multi-tap T9 state machine
         preview   = Keypad_GetPreview() // currently highlighted letter
        IF preview changed AND FSM idle:
                show 'X: .-.' on LCD line 2 AND update 7-seg
        IF confirmed == '*': g_flag_sw5_pressed = 1 (mode toggle)
          IF confirmed == valid letter or digit:
                 push onto circular queue
                 first keypress: clear 'Morse Encoder' from LCD line 1
                add confirmed char to LCD line 1 scroll
        IF confirmed == '#': push word-gap marker '#' to queue
        IF g_enc_state == ENC_IDLE AND queue not empty:
                c = queue_pop()
                copy Morse_Encode(c) into g_enc_pattern[]
                g_enc_state = ENC_START_SYMBOL
                show 'X: .-.' on LCD line 2 + 7-seg

TIM2_IRQHandler (every 200 ms — the encode heartbeat):
        clear UIF flag
        ENC_START_SYMBOL: read pattern[idx]; BUZZER_ON, LED_ON -> ENC_SOUND_ON
        ENC_SOUND_ON:     decrement ticks; if done: BUZZER_OFF, LED_OFF -> SOUND_OFF
        ENC_SOUND_OFF:    decrement 1-tick gap; advance idx -> ENC_START_SYMBOL
        ENC_INTER_LETTER: decrement 3-tick gap; if done -> ENC_IDLE
        ENC_INTER_WORD:   decrement 7-tick gap; if done -> ENC_IDLE
```

**Pseudocode – Decode Mode**

```
MAIN LOOP (decode mode):
        Keypad_Read()  // always called — detects '*' for mode toggle
        IF g_flag_sw4_pressed:   // EXTI15_10_IRQHandler set on FALLING edge
                LED0_ON()
                show current buffer + underscore on LCD line 2
        IF g_flag_sw4_released:  // EXTI15_10_IRQHandler set on RISING edge
                LED0_OFF()
                dur = g_sw4_press_duration_ms
                symbol = (dur < 300ms) ? '.' : '-'
                append symbol to g_dec_buf
```

```
                show accumulated pattern on LCD line 2
                brief beep to confirm symbol received
        IF g_flag_tim6_fired:    // TIM6_DAC_IRQHandler set after 600ms silence
                letter = Morse_Decode(g_dec_buf)
                IF letter != '?':
                        first decode: clear 'Morse Decoder' from LCD line 1
                        show letter on LCD line 1 scroll + 7-seg; success beep
                ELSE: show '?' on LCD line 2 + double error beep
                clear g_dec_buf; clear LCD line 2

EXTI15_10_IRQHandler (PC13, active LOW, both edges):
        FALLING edge (pin LOW  = button pressed):
                TIM3_Reset_And_Start(); TIM6_Stop(); g_flag_sw4_pressed = 1
        RISING  edge (pin HIGH = button released):
                TIM3_Stop(); g_sw4_press_duration_ms = TIM3->CNT
                g_flag_sw4_released = 1; TIM6_Start_OneShot()

TIM6_DAC_IRQHandler: g_flag_tim6_fired = 1
```

**Page 7 contains the flowchart/finite state machine for the Morse Encode mode. Page 8 contains the flowchart/finite state machine for the Morse Decode mode. Please see below.**

**Figure 3.1, Encode FSM.** A flowchart that follows the cycle of the Encode pseudocode.

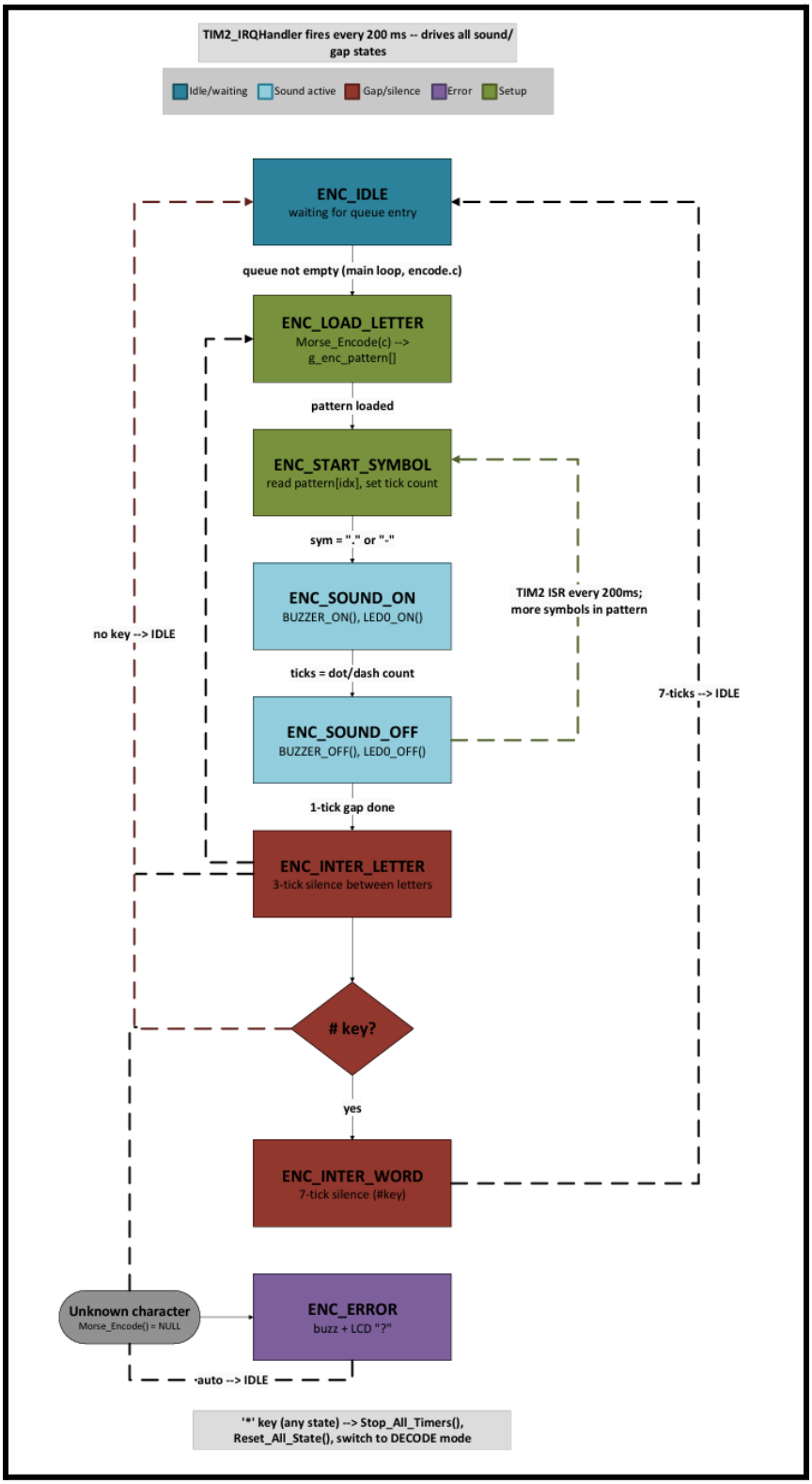
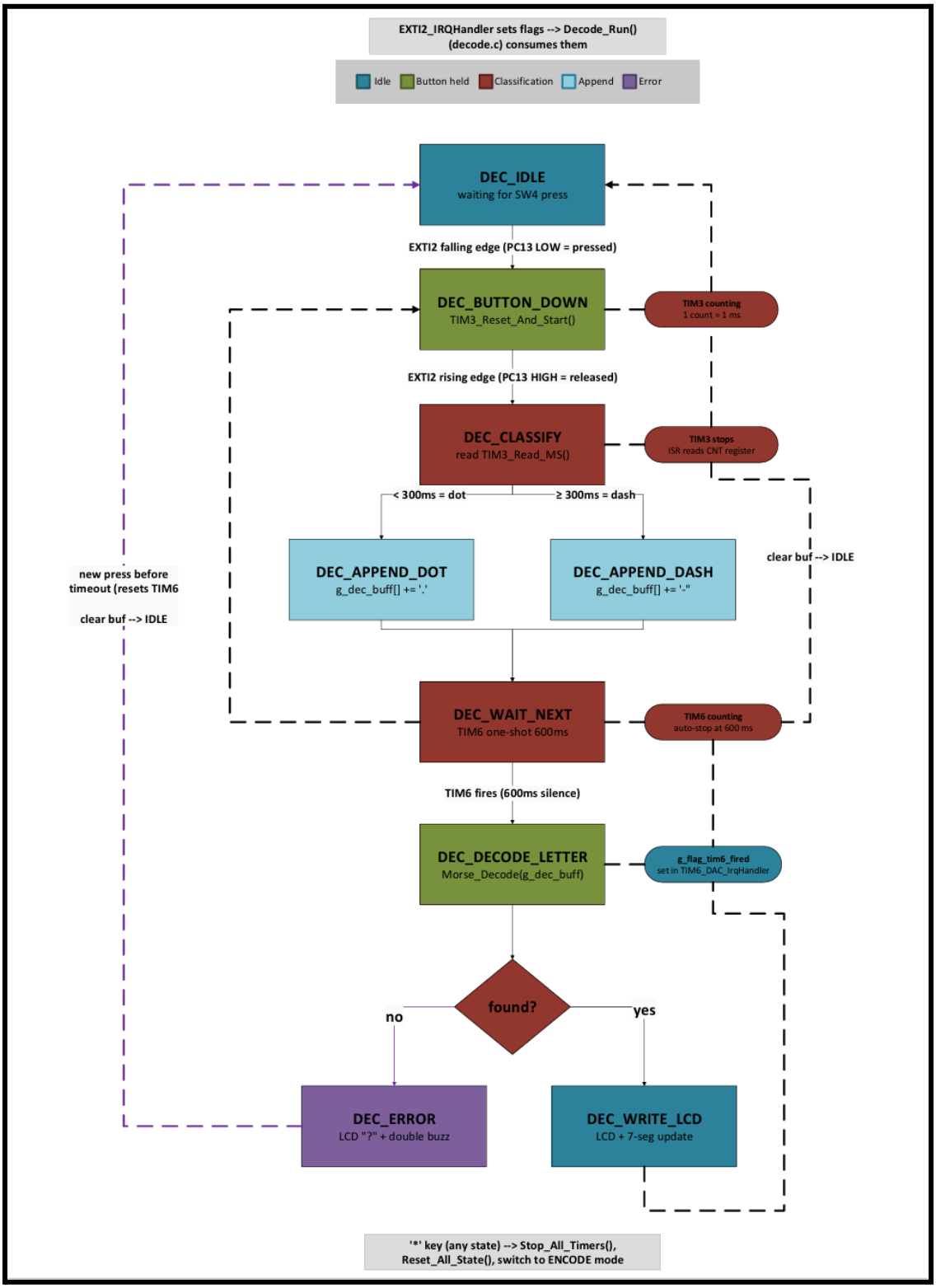


**Figure 3.2, Decode FSM.** A flowchart that follows the cycle of the Decode pseudocode.



**Multi-Tap Key Mapping**

| KEY | CYCLE (TAP TO ADVANCE) | NOTES |
|---|---|---|
| 1 | E F G 1 | 4 taps to reach digit 1; wraps back to E |
| 2 | H I J 2 | 4 taps to reach digit 2 |
| 3 | K L M 3 | 4 taps to reach digit 3 |
| 4 | N O P 4 | 4 taps to reach digit 4 |
| 5 | Q R S 5 | 4 taps to reach digit 5 |
| 6 | T U V 6 | 4 taps to reach digit 6 |
| 7 | W X Y Z 7 | 5 taps to reach digit 7 |
| 8, 9, 0 | Digit direct (1 tap) | No letters on these keys |
| A, B, C, D | Letter direct (1 tap) | No multi-tap |
| * | Mode toggle | Confirms any pending letter first, then switches mode |
| # | Word-gap (7-tick silence) | Confirms any pending letter first |

**Module Interaction**

The codebase is split into 13 source files

- gpio.c handles all pin configuration
- timers.c owns TIM2/3/6 configuration and start/stop logic
- exti.c configures the SYSCFG multiplexer routing PC13 to EXTI line 13 and enables the interrupt in NVIC
- stm32l4xx_it.c contains all four ISRs and is the only file that directly changes FSM state variables from interrupt context
- encode.c and decode.c are called from the main loop and consume ISR-set flags
- keypad.c implements the full multi-tap state machine
- morse_table.c provides the bidirectional Morse lookup used by both modes
- keypad.c, lcd.c, and seg7.c are hardware drivers with no knowledge of the FSM states

## B. Interface Design

| PIN | DIRECTION | COMPONENT | PURPOSE |
|---|---|---|---|
| PA1 | Output | LED0 | Primary Morse indicator – on during sound |
| PA5 | Output | Shift clock | Shared clock for 7-seg and LCD shift registers |

| | | | |
|---|---|---|---|
| PA10 | Output | LCD latch | Strobe to commit data to LCD shift register |
| PB1-4 | Output | Keypad columns | Column scan outputs – driven HIGH one at a time |
| PB5 | Output | SR data | Serial data for 7-seg and LCD (shared) |
| PB8-11 | Input | Keypad rows | Row read inputs – no internal pull |
| PC9 | Output | Buzzer | Audio Morse output – toggled to produce tone |
| PC10 | Output | 7-seg latch | Strobe to commit data to 7-segment display |
| PC13 | Input | Blue button | Decode dot/dash input – EXTI line 13, both edges, pull-up, active LOW |

The LCD and 7-segment display share a two-wire shift register bus (PB5 data, PA5 clock) with separate latch pins (PA10 for LCD, PC10 for 7-segment). 16 bits are clocked in with each write and the appropriate latch is pulsed to commit data. Mode toggle uses no dedicated hardware button — the '*' keypad key sets g_flag_sw5_pressed directly in software, avoiding any conflict with the EDUBase push buttons which share the PB8-PB11 row lines with the keypad matrix.

## C. Microcontroller Resource Utilization

**Timers**

| TIMER | CONFIG | MODE | ROLE IN PROJECT |
|---|---|---|---|
| TIM2 | PSC = 3999, ARR = 199 | Periodic ISR, 200ms | Drives all encode output – SOUND_ON/GAP states |
| TIM3 | PSC = 3999, ARR = 0xFFFF | Free-running counter | Measures PC13 press duration in ms for dot/dash classification |
| TIM6 | PSC = 3999, ARR = 599, OPM | One-pulse (one-shot) | 600ms decode timeout – fires ISR after user pauses input |

All three timers use PSC=3999, dividing the 4 MHz MSI clock to 1000 Hz (1 ms per count). TIM2 uses ARR=199 for 200 ms overflow. TIM3 uses ARR=0xFFFF and counts freely; the ISR reads CNT directly on button release. TIM6 uses ARR=599 for 600 ms and is configured in One Pulse Mode (OPM) so it stops automatically after firing.

**Interrupts**

- TIM2_IRQn (IRQ 28): NVIC->ISER[0] bit 28. Priority 2. Fires every 200 ms to advance encode FSM.
- TIM6_DAC_IRQn (IRQ 54): NVIC->ISER[1] bit 22. Priority 2. Fires 600 ms after last button release to trigger decode.

- EXTI15_10_IRQn (IRQ 40): NVIC->ISER[1] bit 8. Priority 0 (highest). Both edges on PC13 (blue button). FALLING = pressed (start TIM3, stop TIM6). RISING = released (stop TIM3, read CNT, start TIM6). Handler: EXTI15_10_IRQHandler.

**I/O Ports**

- Port A (RCC->AHB2ENR bit 0): LED 0, PA5 shift clock, PA10 LCD latch
- Port B (bit 1): PB1-4 keypad columns, PB5 shift data, PB8-11 keypad rows
- Port C (bit 2): PC9 buzzer, PC10 7-seg latch

**Memory**

- Flash (1 MB at 0x08000000): program code, Morse lookup table, multi-tap letter tables, initialized constants
- SRAM (96 KB at 0x20000000): letter queue (32 bytes), decode buffer (8 bytes), multi-tap state variables (mt_key, mt_count, mt_idle), all FSM state variables, and the stack
- No external RAM or ADC used in this project

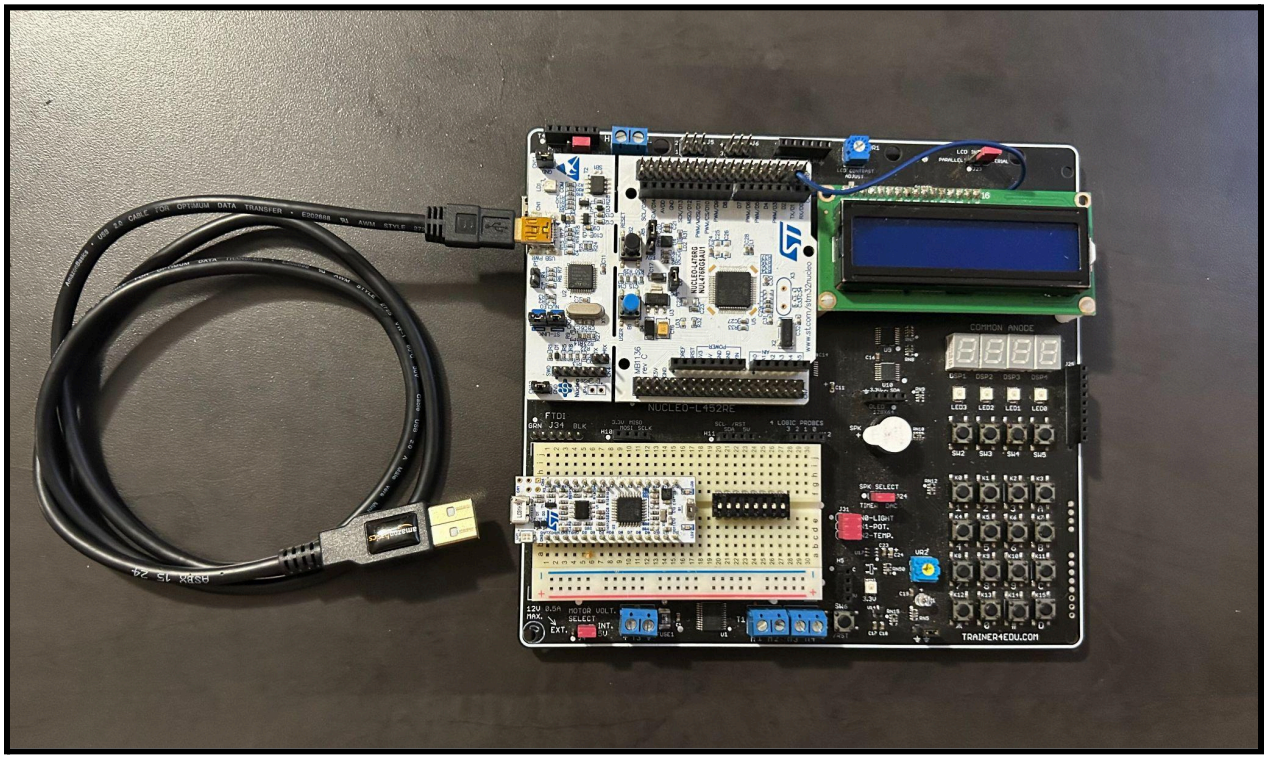
## D. Software

The software is written in C and organized across 13 source files plus one central header. All Morse timing is hardware-driven — no Delay() calls exist in any ISR or in the encode/decode FSM paths. Software delay is used only in the LCD initialization sequence, keypad debounce, and brief confirmation beeps, which occur during non-time-critical moments.

The ISR design follows the flag-based pattern required for correct embedded concurrency: ISRs are intentionally minimal, setting volatile flags and updating shared counters. The main loop reads those flags and performs all LCD writes, Morse table lookups, and display updates. This prevents slow shift-register operations (which require Delay() calls for timing) from blocking inside an ISR and causing missed interrupts or timing errors.

The multi-tap input scheme in keypad.c uses a state machine with three variables: mt_key (which digit key is being tapped), mt_count (how many times it has been tapped), and mt_idle (how many main-loop iterations have elapsed since the last tap). When mt_idle reaches MULTITAP_TIMEOUT_LOOPS (default 800 iterations, approximately 8 seconds), the current selection is confirmed automatically. Pressing a different key confirms the current selection and starts a new one. The '*' key confirms any pending selection and additionally sets g_flag_sw5_pressed, which the main loop reads to switch modes. The scan function (scan_raw) performs a full column-by-column scan before accepting any keypress, which prevents false triggers from the EDUBase push buttons that pull keypad row pins HIGH without a column being driven.

## E. Hardware Photos




This image is the NUCLEO-L476RG seated in the EDUBase V2, showing the keypad, LCD, 7-segment display, and more. Everything connects automatically through the Morpho headers when you seat the NUCLEO-L476RG into the EDUBase V2; you do not need to manually wire anything for the on-board components (LEDs, buzzer, keypad, LCD, 7-segment, and buttons). Only thing to verify physically: make sure jumper J23 on the EDUBase is set to the serial (shift register) position for the LCD.

# IV. ANALYSIS

## A. Functionality and Testing

Testing was conducted in ten phases, each isolating one hardware layer before combining it with others. This bottom-up approach ensured that when a failure occurred, the responsible layer could be identified without ambiguity.

| TEST | EXPECTED | RESULT |
|---|---|---|
| Phase 1 – GPIO: LED blink | LED0 (PA1) blinks at visible rate | Pass |
| Phase 1 – GPIO: Buzzer | Audible click at each Delay(200) toggle | Pass |
| Phase 2 – LCD init and string | LCD shows “Hello ECE3130” | Pass |
| Phase 3 – 7-segment: digit ‘5’ | Digit 5 on display 2 | Pass |
| Phase 4 – Keypad: any key | Key character on LCD line 2 | Pass |
| Phase 5 – TIM2 blink LED | LED0 toggles every 200ms (2.5 Hz) | Pass |
| Phase 6 – EXTI + TIM3 classify | Short press shows DOT, long press DASH | Pass |
| Phase 7 – TIM6 timeout | TIMEOUT after 600ms of silence | Pass |
| Phase 8 – Encode ‘S’, tap key 5 three times | Three equal short beeps | Pass |
| Phase 8 – Encode ‘O’, tap key 4 two times | Three equal long beeps | Pass |
| Phase 8 – Encode word “SOS” | Full SOS pattern with inter-letter gaps | Pass |
| Phase 9 – Decode dot dot dot | LCD displays ‘S’ | Pass |
| Phase 9 – Decode dash dash dash | LCD displays ‘O’ | Pass |
| Phase 9 – Decode unknown pattern | LCD shows ‘?’ + double error beep | Pass |
| Phase 10 – Mode switch | Clean switch (either way), no corrupted state | Pass |

Several issues were encountered and resolved during development. First, the original ISR file (stm32l4xx_it.c) was missing from the Keil project entirely — without it no interrupts fired, causing nothing to respond. This was resolved by adding the file to Source Group 1 in the Keil project pane. Second, the function name mismatch between System_Clock_Config() (defined in system.c) and SystemClock_Config() (called in main.c) caused a linker conflict; corrected by aligning both to System_Clock_Config(). Third, EDUBase push buttons that were initially considered for mode toggle were found to share the PB8-PB11 keypad row pins, causing spurious mode switches when the top

keypad row was pressed. This was resolved by implementing mode toggle entirely via the '*' keypad key in software. Fourth, the multi-tap auto-confirm timeout was too short (50 iterations), causing rapid double-taps to register as two separate letters rather than advancing the selection; corrected by increasing to 800 iterations. Fifth, LCD line 1 was being overwritten by the first keypress, erasing the startup message unexpectedly; fixed by implementing a first_key flag that clears line 1 only on the very first confirmed character.

**See GitHub folder "Testing and Demonstration" for testing photos, testing code, and full-functionality demonstration.**

## B. Required Discussion Questions

**Question 1: Explain how your systems design considers public health, safety, and welfare. Then suggest specific design modifications that would better address these concerns.**

While currently an educational tool, deploying this system as a real-world emergency beacon necessitates critical safety enhancements. To improve accessibility for uses with motor impairments, the fixed 300 ms dot/dash threshold should be replaced with a user-adjustable calibration mode. From a safety perspective, replacing simple buzzer toggling with PWM-controlled amplitude would allow the volume to adapt to the environment, preventing it from being too quiet in loud areas or startling in silent ones. Finally, to ensure reliability, the STM32's Independent Watchdog (IWDG) must be implemented to automatically reset the system in the event of a software hang, a standard requirement for any safety-critical application.

**Question 2: Explain how your system's design considers global, cultural, social, environmental, and economic factors. Then suggest modifications to better address these factors.**

Although the system utilizes Morse code as a global standard, its cultural inclusivity could be improved by extending the character set to include international prosigns, punctuation, and regional variants beyond the basic alphabet. Economically, moving from a $20 NUCLEO development board to a bare STM32L072 microcontroller would reduce costs to under $2, making the device viable for mass-distributed emergency kits in developing regions. Furthermore, environmental impact can be minimized by leveraging the STM32's low-power stop modes. This would reduce power consumption from milliamperes to microamperes, significantly extending battery life and reducing the waste associated with battery replacement.

# V. DESCRIPTION OF THE TEAMWORK EXPERIENCE

## A. Collaborative and Inclusive Environment

We established a collaborative environment from the outset by having an initial planning meeting to review the project guidelines together and agree on the overall architecture before writing any code. Both team members reviewed the FSM diagrams and agreed on the state definitions and timer assignments before implementation began. Disagreements on design choices, such as whether to use polling or interrupts for button input, were resolved through discussion referencing the lecture materials, with both members contributing to the final decision to use EXTI interrupts for all button input.

The team used a shared GitHub repository with descriptive commit messages so that both members could review each other's work asynchronously. Code review comments were exchanged before any major module was merged into the main branch, ensuring both members understood the full codebase.

## B. Leadership and Goal Setting

Noble took the lead on hardware integration and the low-level driver files (gpio.c, timers.c, exti.c, keypad.c), establishing the register-direct coding style and ensuring all initialization sequences matched given examples. Teagan led the FSM logic design, writing the encode and decode state machines and the Morse lookup table, and was primarily responsible for testing and documentation.

Both members set clear weekly milestones aligned with the two-week project plan: days 1-2 for hardware verification, days 3-4 for TIM2 integrations, days 5-6 for full encode mode, days 7-8 for EXTI and TIM3, and so on. These milestones were reviewed together at the start of each work session.

## C. Task Planning and Distribution

Tasks were distributed by hardware layer and software module to minimize merge conflicts and allow parallel development. Teagan owned all configuration and code files, though they were created and edited jointly on her machine during paired programming sessions.

When the TIM3/EXTI integration took longer than anticipated in days 7-8, the team redistributed tasks so Noble completed the LCD scroll logic while Teagan debugged the EXTI falling edge classification. This flexibility allowed the project to remain on schedule despite the setback. Both team members attended all demo preparation sessions and can explain any part of the design.

---

**GitHub Link to Project –**
https://github.com/ttobias28/microcomputer-morse-code-encoder-and-decoder/tree/main?tab=readme-ov-file